Escola Superior de Tecnologia e Gestão

Politécnico de Coimbra

Licenciatura em Engenharia Informática

Tecnologias e Aplicações Móveis

Ano Letivo 2026/2027

Análise e Planeamento:

SmartAccess

Sistema de Controlo de Acesso e Segurança de Sala Técnica

Elaborado em: 22/10/2026

David Mendes - a2023134616

Dinis Jesus - a2023153073

Rafael Ferreira - a2022158179

Índice

1. Introdução **1**

2. Definição do Projeto **1**

2.1. Título do Projeto 1

2.2. Objetivo do Sistema 1

2.3. Sensores e Atuadores 2

3. Arquitetura e Tecnologias **2**

3.1. Plataforma Embebida 2

3.2. Arquitetura do Sistema 3

3.3. Stack Tecnológica 4

3.3.1. Backend 4

3.3.2. Base de Dados 5

3.3.3. Frontend 5

4. Cronograma **6**

4.1. Fases do Projeto 6

4.2. Milestones 6

4.3. Distribuição de Tarefas 7

5. Conclusão **7**

6. Referências **8**

1. Introdução

O controlo de acessos físicos a espaços que guardam equipamento valioso ou sensível — salas de servidores, laboratórios ou arrecadações técnicas — continua, em muitas escolas e pequenas organizações, a depender exclusivamente de chaves físicas. Esta abordagem não permite saber quem entrou, a que horas, nem impedir o acesso fora de horário, e a perda de uma chave obriga à substituição da fechadura.

Este documento apresenta a análise e o planeamento do **SmartAccess**, um sistema de controlo de acesso de baixo custo baseado em cartões RFID, desenvolvido no âmbito da unidade curricular de Tecnologias e Aplicações Móveis. O sistema integra um dispositivo embebido instalado junto à porta, um backend responsável pela lógica de autorização, uma base de dados relacional para registo de utilizadores e acessos, e uma aplicação Android, desenvolvida em Kotlin, a partir da qual o administrador gere todo o sistema.

O documento está organizado de acordo com os requisitos do enunciado: definição do projeto, arquitetura e tecnologias, cronograma e roadmap. Inclui ainda uma secção dedicada aos riscos e desafios antecipados, com as respetivas estratégias de mitigação, por se considerar essencial para um planeamento realista.

2. Definição do Projeto

2.1. Título do Projeto

- **Título do Projeto:** SmartAccess — Sistema de Controlo de Acesso e Segurança de Sala Técnica

2.2. Objetivo do Sistema

O objetivo do SmartAccess é substituir a chave física de uma sala técnica por um sistema de controlo de acesso inteligente, auditável e gerível remotamente, construído com componentes acessíveis. Cada utilizador autorizado recebe um cartão (ou porta-chaves) RFID. Ao aproximá-lo do leitor instalado junto à porta, o dispositivo embebido envia a identificação do cartão ao backend, que verifica se este está ativo e se o acesso é permitido naquele dia e hora. Em caso afirmativo, a tranca é aberta; caso contrário, o acesso é negado. Em ambos os casos, a tentativa fica registada.

O sistema resolve, assim, três problemas concretos:

- **Falta de rastreabilidade:** todos os acessos e tentativas negadas ficam registados com data, hora, cartão e resultado, permitindo auditorias.
- **Ausência de restrições horárias:** cada cartão pode ter dias e horários de acesso definidos, sendo negado fora desses períodos.
- **Gestão rígida de credenciais:** um cartão perdido é revogado de imediato na aplicação, sem necessidade de trocar fechaduras.

Para além disso, o sistema monitoriza a zona junto à porta através de um sensor de distância ultrassónico e gera alertas em situações anómalas, enviados como notificações para o telemóvel do administrador:

- Presença detetada junto à porta fora do horário permitido;
- Várias tentativas de acesso negadas consecutivas (possível tentativa de utilização indevida);
- Dispositivo embebido sem comunicação com o servidor (offline).

O sistema prevê dois perfis de utilizador: o **administrador**, que aprova e revoga cartões, define permissões, consulta todos os registos, recebe alertas e pode abrir a porta remotamente; e o **utilizador**, que pode consultar o histórico dos seus próprios acessos.

2.3. Sensores e Atuadores

Foram selecionados componentes simples, de baixo custo e presentes nos kits de eletrónica habitualmente disponíveis na escola, de forma a reduzir o risco técnico e o custo do protótipo. O estado da porta (aberta/fechada) é simulado através de dois LEDs, uma vez que o protótipo não inclui uma porta física.

Componente	Qtd.	Tipo	Função e justificação
Arduino UNO WiFi Rev2	1	Controlador	Plataforma embebida com Wi-Fi integrado. Lê os sensores, comunica com o backend e controla os atuadores (justificação na secção 3.1).
Leitor RFID RC522 + cartões/porta-chaves	1 + 4	Sensor	Lê o UID dos cartões MIFARE de 13,56 MHz que identificam cada utilizador. Muito barato, comunica por SPI e tem biblioteca madura (MFRC522).
Sensor ultrassónico HC-SR04	2	Sensor	Mede a distância ao objeto mais próximo. Quando a distância fica abaixo do valor de referência (sala/porta vazia), há uma pessoa presente, o que permite gerar alertas de intrusão fora do horário. Funciona a 5 V e usa apenas 2 pinos digitais (Trig e Echo).
Servo motor SG90	1	Atuador	Simula a tranca da porta. Controlado por PWM e alimentado a 5 V, evita a fonte de alta corrente que um solenoide exigiria.
LED verde	1	Atuador	Indica porta aberta / acesso concedido.
LED vermelho	1	Atuador	Indica porta fechada / acesso negado (estado por omissão).
Display LCD 16x2 com módulo I2C	1	Atuador	Mostra mensagens ao utilizador ("Acesso concedido", "Acesso negado", "Sem ligação"). O I2C ocupa apenas 2 pinos.
Buzzer ativo	1	Atuador	Emite sinais sonoros distintos para acesso concedido, acesso negado e alerta.
Material de suporte	—	—	Breadboard, fios de ligação, 2 resistências de 220 Ω (LEDs), fonte de alimentação externa de 5 V (servo) e cabo USB.

Tabela 1 – Componentes de hardware

Ignorar tudo o que fala de intrusão

Vamos usar 2 sensores para confirmar o n de entradas ENTRADA A ENTRADA B

3. Arquitetura e Tecnologias

3.1. Plataforma Embebida

A plataforma escolhida é o **Arduino UNO WiFi Rev2**. No SmartAccess, a decisão de conceder ou negar um acesso é tomada no backend, onde estão as permissões e os registos. O dispositivo embebido tem, portanto, um papel simples e bem delimitado: ler o cartão e os sensores, enviar os dados ao servidor

e executar a resposta (abrir a tranca, acender o LED, mostrar a mensagem). Para esta função, um microcontrolador é suficiente e mais adequado do que um computador completo como o Raspberry Pi.

Critério	Arduino UNO WiFi Rev2	Raspberry Pi
Custo	Mais baixo; disponível nos kits da escola	Mais elevado (placa + cartão SD + fonte)
Consumo de energia	Muito baixo, adequado a funcionamento contínuo	Superior, por correr um sistema operativo completo
Arranque	Instantâneo após falha de energia	Demora a arrancar o sistema operativo
Fiabilidade	Sem sistema de ficheiros; não corrompe com cortes de energia	Risco de corrupção do cartão SD em cortes de energia
Processamento	Suficiente para ler sensores e fazer pedidos HTTP	Muito superior, mas desnecessário neste projeto
Conectividade	Wi-Fi integrado (módulo NINA)	Wi-Fi integrado

Tabela 2 – Comparação das plataformas embebidas

Uma porta de acesso é um equipamento que tem de estar sempre disponível e recuperar sozinho após uma falha de energia, pelo que o arranque instantâneo e a ausência de cartão SD são vantagens decisivas. O Raspberry Pi só se justificaria se o processamento tivesse de ser feito localmente (por exemplo, reconhecimento facial com câmara), o que não é o caso.

3.2. Arquitetura do Sistema

O sistema segue uma arquitetura cliente-servidor em três camadas, em que o backend é o único componente que acede à base de dados. Tanto o dispositivo embebido como a aplicação Android comunicam exclusivamente com o backend através de uma API REST.

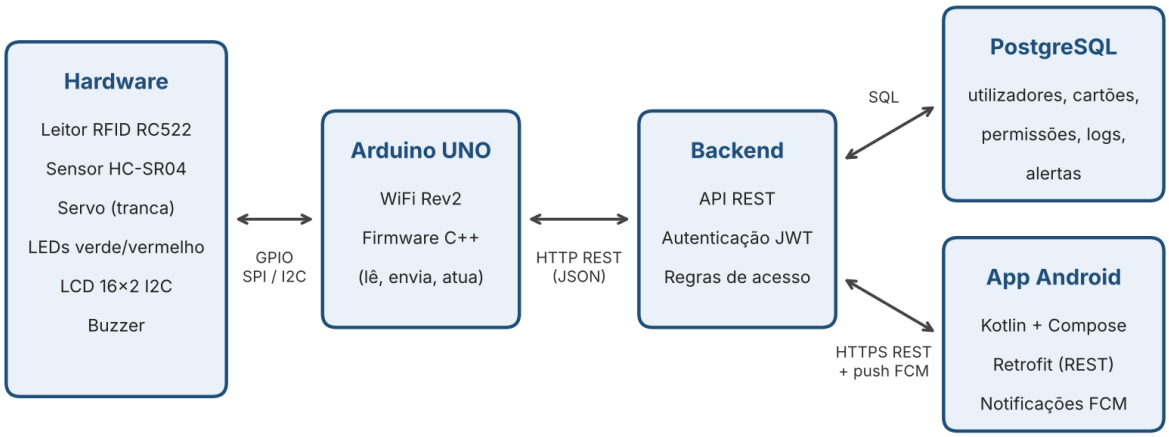


Figura 1 – Arquitetura geral do SmartAccess

Fluxo de acesso por cartão:

1. O utilizador aproxima o cartão do leitor RC522.
2. O Arduino lê o UID e envia-o ao backend num pedido HTTP com formato JSON, identificando-se com a sua chave de API.

3. O backend verifica na base de dados se o cartão existe, se está ativo e se o dia e a hora atuais estão dentro das suas permissões.
4. O backend regista a tentativa e responde com "concedido" ou "negado" e o motivo.
5. Se concedido, o Arduino abre o servo, acende o LED verde, emite um sinal sonoro e mostra a mensagem no LCD durante 5 segundos, voltando depois ao LED vermelho. Se negado, mostra a mensagem de erro e emite um sinal sonoro diferente.

3.3. Stack Tecnológica

3.3.1. Backend

- **Tecnologia:** API REST, desenvolvida em Python, ligada à base de dados PostgreSQL.
- **Justificação:** uma API REST define um conjunto de endpoints claros, acessíveis por HTTP e com mensagens em formato JSON, que podem ser usados da mesma forma pelo Arduino e pela aplicação Android. Por ser independente do cliente, permite desenvolver e testar o firmware e a aplicação em paralelo. O backend é o único componente com acesso ao PostgreSQL, o que centraliza a lógica de negócio e protege os dados.
- **Funções principais:**
  - Receber os pedidos de acesso do dispositivo e decidir se são concedidos, com base no estado do cartão e nas permissões horárias;
  - Registar todos os acessos, tentativas negadas e eventos do dispositivo;
  - Gerir utilizadores, cartões e permissões (lógica de negócio);
  - Gerar alertas (presença fora de horas, tentativas negadas consecutivas, dispositivo offline) e enviá-los por notificação push;
  - Gerir a fila de comandos de abertura remota;
  - Autenticar os utilizadores da aplicação e o dispositivo embebido.
- **Protocolos de comunicação:** HTTP REST com mensagens JSON entre o Arduino e o backend; HTTPS REST entre a aplicação e o backend; Firebase Cloud Messaging (FCM) para notificações push.
- **Segurança:** autenticação dos utilizadores com JWT (tokens com expiração), palavras-passe guardadas com hash bcrypt e chave de API própria para o dispositivo embebido.

Método	Endpoint	Descrição
POST	/api/auth/login	Autenticação na aplicação; devolve o token JWT
POST	/api/access	O dispositivo envia o UID lido; o backend responde concedido/negado
GET	/api/device/commands	O dispositivo consulta comandos pendentes (abertura remota)
POST	/api/device/events	O dispositivo comunica eventos (presença detetada, heartbeat, confirmação de comandos)
POST	/api/door/unlock	A aplicação pede a abertura remota da porta
GET	/api/logs	Histórico de acessos com filtros (data, utilizador, resultado)
GET / PATCH	/api/cards	Listar cartões, aprovar, associar a utilizador ou revogar

Método	Endpoint	Descrição
PUT	/api/cards/{id}/permissions	Definir os dias e horários de acesso de um cartão
GET / PATCH	/api/alerts	Listar alertas e marcá-los como resolvidos

Tabela 3 – Principais endpoints da API REST

3.3.2. Base de Dados

- **Tipo:** relacional (SQL) — PostgreSQL.
- **Justificação:** os dados do sistema são fortemente relacionados (um utilizador tem cartões, um cartão tem permissões, um registo refere-se a um cartão), pelo que um modelo relacional com chaves estrangeiras garante a sua integridade. O PostgreSQL é gratuito, robusto e suporta bem consultas por intervalos de datas, necessárias para filtrar o histórico de acessos.
- **Função:** armazenar utilizadores, cartões e permissões; manter o registo permanente de todos os acessos e eventos; guardar alertas e comandos pendentes.
- **Registos imutáveis:** a tabela de registos de acesso apenas aceita inserções, não permitindo alterar nem apagar registos através da API, garantindo a fiabilidade das auditorias.

Tabela	Principais campos
utilizadores	id, nome, email, password_hash, perfil (admin | utilizador), criado_em
cartoes	id, uid (único), utilizador_id, estado (pendente | ativo | revogado), criado_em
permissoes	id, cartao_id, dia_semana, hora_inicio, hora_fim
registos_acesso	id, cartao_uid, utilizador_id, origem (cartão | remoto), resultado, motivo, data_hora
alertas	id, tipo, descricao, data_hora, resolvido
dispositivos	id, nome, api_key_hash, ultimo_contacto
comandos	id, dispositivo_id, tipo, estado (pendente | executado), criado_por, criado_em

Tabela 4 – Modelo de dados

3.3.3. Frontend

- **Tecnologia:** aplicação Android nativa em Kotlin, com interface em Jetpack Compose e arquitetura MVVM.
- **Bibliotecas:** Retrofit e OkHttp (comunicação com a API), Kotlin Coroutines/Flow (operações assíncronas), Firebase Cloud Messaging (notificações) e DataStore (armazenamento seguro do token).
- **Descrição da interface:**
  - **Login:** autenticação com email e palavra-passe;
  - **Painel principal:** estado da porta (aberta/fechada), estado do dispositivo (online/offline), último acesso registado e botão "Abrir porta" (apenas para administradores, com pedido de confirmação);
  - **Registos:** lista de acessos atualizada automaticamente, com filtros por data, utilizador e resultado;
  - **Cartões:** cartões pendentes (lidos mas desconhecidos), que o administrador aprova e associa a um utilizador; cartões ativos, que podem ser revogados; definição dos horários de acesso de cada cartão;

- **Alertas:** lista de alertas com possibilidade de os marcar como resolvidos; os novos alertas chegam também como notificação push.

4. Cronograma

4.1. Fases do Projeto

O desenvolvimento foi organizado em sete fases. Sempre que possível, as fases decorrem em paralelo (por exemplo, a prototipagem do hardware e o desenvolvimento do backend), tirando partido da divisão do trabalho pelos elementos do grupo. A documentação automática da API permite que a aplicação Android comece a ser desenvolvida antes de a integração com o hardware estar concluída.

Fase	Período	Tarefas principais
Fase 0 – Planeamento e validação	22/10 – 30/10	Definição de requisitos, componentes e tecnologias; elaboração deste documento; validação do projeto pelo docente; criação do repositório Git.
Fase 1 – Prototipagem do hardware	02/11 – 13/11	Montagem do circuito na breadboard; testes individuais de cada componente (RC522, HC-SR04, servo, LEDs, LCD, buzzer); calibração da distância de referência; firmware base com leituras pela porta série; ligação Wi-Fi.
Fase 2 – Backend e base de dados	02/11 – 20/11	Criação do esquema em PostgreSQL; implementação da API REST ligada ao PostgreSQL; autenticação JWT; regras de permissões horárias; testes com Postman.
Fase 3 – Integração hardware ↔ backend	16/11 – 27/11	Envio do UID e receção da decisão; consulta de comandos remotos; envio de eventos e heartbeat; modo de funcionamento sem rede.
Fase 4 – Aplicação Android	16/11 – 11/12	Ecrãs de login, painel, registos, cartões e alertas; integração com a API via Retrofit; abertura remota; notificações push.
Fase 5 – Testes e afinação	07/12 – 15/12	Testes ponta a ponta; testes de falha de rede e de energia; testes de segurança (tokens, chave de API, acessos fora de horário); correção de erros.
Fase 6 – Relatório final e entrega	14/12 – 18/12	Relatório final, preparação da demonstração e entrega. Os dias 16 a 18/12 servem de margem para imprevistos.

Tabela 5 – Fases do projeto

4.2. Milestones

- **30/10/2026 – M0:** Projeto validado pelo docente e documento de planeamento submetido;
- **13/11/2026 – M1:** Protótipo de hardware montado, com todos os sensores e atuadores a funcionar e ligação Wi-Fi estabelecida;
- **20/11/2026 – M2:** API REST e base de dados funcionais e testadas, incluindo autenticação e regras de permissões;
- **27/11/2026 – M3:** Fluxo completo de acesso por cartão a funcionar, com decisão no backend e registo na base de dados;
- **11/12/2026 – M4:** Aplicação Android concluída, com gestão de cartões, registos, alertas e abertura remota;
- **15/12/2026 – M5:** Sistema totalmente integrado e testado;

- **18/12/2026 – M6:** Entrega final do projeto.

Justificação dos prazos:

- O hardware e o backend avançam em paralelo desde o início, permitindo iniciar a integração a meio de novembro;
- A aplicação Android tem o período mais longo (quatro semanas), por ser a componente com mais ecrãs e funcionalidades;
- A fase de testes sobrepõe-se ao final do desenvolvimento, para que os erros sejam detetados cedo;
- São reservados três dias de margem antes da entrega para imprevistos;
- As datas serão ajustadas caso a data de entrega definida pelo docente seja diferente.

4.3. Distribuição de Tarefas

Cada elemento do grupo é responsável por uma camada do sistema, sendo a integração e os testes realizados em conjunto. Esta divisão permite trabalhar em paralelo e garante que cada componente tem um responsável claro.

Responsável	Área
Elemento 1	Hardware e firmware do Arduino (montagem, leitura de sensores, comunicação HTTP, modo sem rede)
Elemento 2	Backend e base de dados (API, regras de negócio, autenticação, alertas)
Elemento 3	Aplicação Android (interface, integração com a API, notificações)
Todos	Integração, testes, relatório e demonstração final

Tabela 6 – Distribuição de tarefas (num grupo de dois elementos, as áreas do backend e da aplicação são partilhadas)

5. Conclusão

O planeamento apresentado demonstra que o SmartAccess é um projeto tecnicamente viável, que cumpre todos os requisitos da unidade curricular: um dispositivo embebido com sensores e atuadores, comunicação em rede, um backend com lógica de negócio própria, uma base de dados relacional e uma aplicação Android em Kotlin.

As escolhas tecnológicas foram feitas tendo em conta a função de cada componente: um Arduino UNO WiFi Rev2 para uma tarefa simples e contínua junto à porta, um backend com API REST onde se concentra a decisão de acesso, o PostgreSQL para garantir a integridade dos registos e uma aplicação Android nativa que permite gerir o sistema e receber alertas em qualquer lugar. O cronograma, com fases em paralelo e milestones semanais, e a identificação antecipada dos riscos permitem um desenvolvimento controlado e a entrega dentro do prazo. Este documento servirá de orientação ao longo do projeto e será atualizado sempre que necessário.

6. Referências

- ARDUINO. Arduino UNO WiFi Rev2 — Documentation. Disponível em: https://docs.arduino.cc/hardware/uno-wifi-rev2/
- BALBOA, M. MFRC522 — Arduino RFID Library. Disponível em: https://github.com/miguelbalboa/rfid
- POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL Documentation. Disponível em: https://www.postgresql.org/docs/
- GOOGLE. Android Developers — Kotlin e Jetpack Compose. Disponível em: https://developer.android.com/
- SQUARE. Retrofit. Disponível em: https://square.github.io/retrofit/
- GOOGLE. Firebase Cloud Messaging. Disponível em: https://firebase.google.com/docs/cloud-messaging